

Primer on Stochastic Optimization Methods with Applications in Financial Engineering

Jenny Petrova

April 29, 2026

Abstract

In this report, we present the general formulation for a stochastic optimization procedure. There exists a vast literature of iterative methods for optimizing an objective function when only noisy observations are available. These methods arise naturally in models where the objective function cannot be evaluated exactly, but rather approximated through random samples. This paper provides a unified overview of stochastic approximation and stochastic gradient-based methods (or "optimizers"), beginning with the classical Robbins–Monro procedure for root-finding and its extension to optimization via stochastic gradient descent (SGD). We then categorize modern improvements to SGD into the following: adaptive learning rate methods, accelerated (momentum-based) methods, and variance reduction techniques. Each category addresses a distinct source of inefficiency in stochastic optimization, including poor conditioning, slow convergence trajectories, and gradient noise (high estimator variance).

Finally, we examine the role of importance sampling as a variance reduction technique, highlighting its connection to stochastic approximation and its relevance in financial engineering problems involving rare-event simulation. We present a numerical example, demonstrating the practical implementation of these methods in determining the optimal sampling distribution for pricing barrier options.

1 Introduction

Stochastic optimization concerns the problem of minimizing an objective function when only noisy observations (or random samples) of the function are available. Formally, we consider problems of the form

$$\min_{x \in \mathbb{R}^d} f(x) := \mathbb{E}[F(x, \xi)],$$

where ξ is a random sample from density p and $F(x, \xi)$ is a loss function that represents a stochastic realization of the objective function. In many applications, the expectation cannot be computed in closed form and must instead be approximated via simulation. In other words, if we cannot directly optimize $F(x, \xi)$, we may instead minimize the expected value, $\mathbb{E}[F(x, \xi)]$, using some unbiased estimates of $F(x, \xi)$. Stochastic approximation methods provide a framework for solving these problems using iterative procedures.

Beginning with the seminal work of Robbins and Monro, these methods construct sequences of estimates that converge to the desired solution using only noisy observations. Over time, this framework has evolved into a broad class of optimizers, including stochastic gradient descent (SGD) and its numerous variants.

The goal of this paper is to organize these developments into a unified framework. In particular, we focus on the following improvements to stochastic gradient methods:

- methods that improve the statistical efficiency of the final estimator,
- methods that adapt the learning rate to account for parameter sensitivity,
- methods that accelerate convergence by modifying the trajectory of iterates,
- methods that improve the quality of the gradient estimator, and
- methods that reduce variance through improved sampling techniques.

In the sections that follow, we develop each of these ideas in detail and illustrate their relevance in financial engineering applications.

2 Stochastic Approximation Framework

Stochastic approximation describes a family of iterative methods designed for root-finding and optimization problems. These methods are particularly useful when exact function evaluations are computationally expensive or impossible to obtain. The basic framework of a stochastic approximation algorithm is a recursion of the form

$$x_{t+1} = x_t - \eta_t Y_t, \quad (1)$$

where x is a parameter of the system, $\{Y_t := Y(x_t)\}$ is a sequence of random variables (collection of noisy observations) corresponding to $\{x_t\}$, and $\{\eta_t\}$ is a sequence of learning rates (step sizes). The goal is to design the recursion (1) so that x_t converges to a desired target value, which we denote as x^* , such that

$$Y(x^*) = \min_x \mathbb{E}[Y_t | x_t]. \quad (2)$$

A typical assumption is that Y_t is an observable, unbiased estimator of some function $f(x_t)$ such that

$$\mathbb{E}[Y_t | x_t] = f(x_t),$$

so that (1) behaves like a deterministic dynamical system.

2.1 Robbins–Monro Procedure

Robbins & Monro [21] introduced the foundational stochastic approximation procedure for finding the root of an unknown function, $f(x)$, that cannot be observed directly. Suppose there exists a unique root x^* such that the equation

$$f(x^*) = 0,$$

where $f(x)$ is non-decreasing and $f'(x) > 0$. Let ξ_t be an unbiased sample of $f(x_t)$ such that

$$\mathbb{E}[\xi_t | x_t] = f(x_t).$$

The Robbins–Monro approach is to generate successive approximations of the form

$$x_{t+1} = x_t - \eta_t \xi_t, \quad (3)$$

where $\eta_1, \eta_2, \dots$ are a sequence of positive step sizes that satisfy the conditions

$$\sum_{n=1}^{\infty} \eta_n = \infty \quad \text{and} \quad \sum_{n=1}^{\infty} \eta_n^2 < \infty. \quad (4)$$

The above conditions ensure $x_t \rightarrow x^*$ in probability under suitable regularity assumptions. Under the classical step size $\eta_t = c/t$, the mean-squared error satisfies $\mathbb{E}[(x_t - x^*)^2] \rightarrow 0$ at a rate of $O(1/t)$; however, the constant c must be chosen carefully relative to $f'(x^*)$, as a poor choice can significantly slow convergence even when the asymptotic rate is preserved.

2.2 Stochastic Gradient Descent

Stochastic Gradient Descent (SGD) is an iterative method that evolved from the Robbins–Monro procedure, designed for optimizing an objective function. Suppose the stochastic optimization problem is to find x^* such that

$$f(x^*) = \min_x \mathbb{E}[F(x, \xi)] \quad (5)$$

where $F(x, \xi)$ is a loss function depending on model parameter x and underlying process ξ . This problem is equivalent to finding the root x^* of $\nabla f(x) = 0$, which is analogous to Robbins–Monro constructed with a gradient estimator. In practice, the exact gradient $\nabla f(x)$ may not be available, so one can instead compute

$$\mathbb{E}[g(x_t, \xi_t) | x_t] = \nabla f(x_t), \quad (6)$$

where $g_t := g(x_t, \xi_t) = \nabla F(x_t, \xi_t)$ is an unbiased estimator of $\nabla f(x_t)$. The SGD update is then given by

$$x_{t+1} = x_t - \eta_t g_t, \quad (7)$$

where at each iteration a single gradient g_t is computed.

Each update moves the parameter in the direction of the global minimizer, and converges almost surely if the objective function f is convex and the step size conditions (4) are satisfied. For a strongly convex f , if a predetermined step size $\eta_t = O(1/t)$ ¹ is chosen, SGD achieves an improved convergence rate of

$$\mathbb{E}[f(x_T) - f(x^*)] = O(1/T),$$

where x_T is the final iterate obtained from the algorithm [20]. These rates assume the gradient estimator g_t has bounded variance, i.e. $\mathbb{E}[\|g_t - \nabla f(x_t)\|^2] \leq C^2$ for some constant $C^2 < \infty$. In more general (non-convex) settings, the iterates typically converge to stationary points, such that

$$\frac{1}{T} \sum_{t=1}^T \mathbb{E}[\|\nabla f(x_t)\|^2] = O\left(\frac{1}{\sqrt{T}}\right).$$

2.3 Kiefer–Wolfowitz Algorithm

Motivated by the Robbins–Monro algorithm, Kiefer & Wolfowitz [11] introduced a gradient-like method for stochastically estimating the optimum of a function. The Kiefer–Wolfowitz method is useful when independent experiments are available to estimate the value of a loss function F at parameter x , but the function is not differentiable or an estimate of the gradient is not available. We assume $\mathbb{E}[F(x, \xi)] = f(x)$ for some observable function F and underlying process ξ of the model. The approach is a finite-difference scheme such that

$$x_{t+1} = x_t - \eta_t \frac{F(x_t + c_t, \xi_t^{(1)}) - F(x_t - c_t, \xi_t^{(2)})}{2c_t}, \quad (8)$$

where $\{c_t\}$ specifies a sequence of perturbation sizes and the observations at $x_t + c_t$ and $x_t - c_t$ are statistically independent (meaning $\xi_t^{(1)}$ and $\xi_t^{(2)}$ are independent random variables). The use of independent observations is essential for unbiasedness, where

$$\mathbb{E}\left[\frac{F(x_t + c_t, \xi_t^{(1)}) - F(x_t - c_t, \xi_t^{(2)})}{2c_t}\right] = \frac{f(x_t + c_t) - f(x_t - c_t)}{2c_t} = f'(x_t) + O(c_t^2) \approx f'(x_t).$$

However, the choice of c_t involves a fundamental bias-variance tradeoff: if c_t is too large, the finite difference is a poor approximation to $f'(x_t)$, whereas if c_t is too small, the noise in F dominates the numerator. As a result, Kiefer–Wolfowitz posed the following conditions: choose $\{\eta_t\}$ and $\{c_t\}$ ² to be positive-valued sequences such that the step size conditions (4) hold and

$$\sum_{t=1}^{\infty} \eta_t c_t < \infty \quad \text{and} \quad \sum_{t=1}^{\infty} \frac{\eta_t^2}{c_t^2} < \infty, \quad (9)$$

where $c_t \rightarrow 0$ as $n \rightarrow \infty$.

2.4 Polyak–Ruppert Averaging

Independently proposed by Ruppert [22] and Polyak [19], Polyak–Ruppert averaging is a technique used to improve the final estimator achieved from a stochastic optimization algorithm. There exists a notable tradeoff between step-size selection and convergence: a fast-decaying learning rate is needed to overcome

¹ $\eta_t = O(1/t)$ indicates the step size is chosen such that it decreases at a rate proportional to $1/t$ as $t \rightarrow \infty$. A popular choice is $\eta_t = 1/t$, which satisfies the Robbins–Monro conditions for step size selection. This was generalized by Fabian [5] to $\eta_t = \alpha/t^\gamma$, for $\gamma \in (1/2, 1]$ and constant $\alpha > 0$. The latter choice preserves the stochastic approximation convergence conditions while allowing greater flexibility in controlling the bias-variance tradeoff of the iterates.

²Kiefer & Wolfowitz suggested perturbation parameters of the form $c_t = 1/\sqrt{t}$.

gradient noise and local convergence, yet a slowly-decaying learning rate is needed to make progress toward the optimal x^* . Rather than relying on the step size to average out noisy observations, the Polyak–Ruppert method instead uses the weighted average of the iterates themselves:

$$\bar{x}_T = \frac{1}{T} \sum_{t=1}^T x_t, \quad (10)$$

where a constant step-size η_t is assumed in the approximation procedure. For a general convex f with step size $\eta_t = O(1/\sqrt{t})$, SGD achieves

$$\mathbb{E}[f(\bar{x}_T) - f(x^*)] = O(1/\sqrt{T}),$$

where $\bar{x}_T$ denotes the running average of iterates [14]. This rate is known to be optimal among first-order stochastic methods for convex functions. Particularly, if $\eta_t \rightarrow 0$ slower than $O(1/t)$ then a better estimate of the final iterate is given by $\bar{x}_T$ in (10).

A practical limitation of averaging is that early iterates, which may be far from x^* , can bias the average and slow down convergence. To mitigate this, a moving window average is often preferred in practice:

$$\bar{x}_T = \frac{1}{M} \sum_{n=T-M+1}^T x_n, \quad (11)$$

for $T > M$. This choice combines the stability benefits of averaging with faster adaptation to the region near x^* .

3 Momentum Methods

Momentum-based techniques are extensions of vanilla SGD used to accelerate training. Originating from the concept of momentum in physics, the goal of these methods is to overcome oscillation of noisy gradients and prolonged convergence of SGD by building "inertia" in a search direction. A common issue with SGD is that previous steps are not taken into consideration at each iteration. Momentum improves on SGD by using an exponentially weighted average of the past gradients to scale the updates at each iteration, hence giving more weight to recent gradients.

3.1 SGD with Momentum

Assume $g_t := g(x_t, \xi_t)$ is an unbiased gradient estimator for a given objective function, as in (6). The update rule for SGD with momentum (SGDM) is given by:

$$m_t = \beta_t m_{t-1} + (1 - \beta_t) g_t, \quad (12)$$

$$x_{t+1} = x_t - \eta_t m_t, \quad (13)$$

where $\beta_t \in [0, 1)$ is the momentum coefficient that determines the relative contribution of the current gradient to earlier gradients in the weight change. SGDM first computes the gradient at the current location and then takes a step in the direction of the exponentially averaged gradient. Observe that taking $\beta_t \equiv 0$ reduces SGDM to vanilla SGD.

The recursion for m_t (12) is an exponential moving average of past gradients, which can be written as:

$$m_t = (1 - \beta_t) \sum_{k=0}^t \beta_t^{t-k} g_k.$$

The momentum parameter smooths the optimization path by using exponential averages to dampen high-variance, noisy updates, thus accelerating convergence in the direction of the minimum.

3.2 SGD with Polyak’s Heavy-Ball Method

Polyak [18] proposed a method which incorporates a momentum term to accumulate information from past iterates. Known as the heavy-ball (HB) method in the literature, the update rule incorporates a “momentum” term $\beta(x_t - x_{t-1})$ to damp oscillations and accelerate convergence:

$$x_{t+1} = x_t - \eta_t g_t + \beta(x_t - x_{t-1}). \quad (14)$$

The overall effect of the momentum parameter is that it allows for larger step sizes η_t and can significantly accelerate convergence in practice, particularly for ill-conditioned problems. Unlike SGDM, which averages the gradients, the heavy-ball method directly uses past parameter updates. While it performs well empirically, convergence is generally limited to strongly convex problems under appropriate parameter choices, and instability may occur otherwise.

3.3 SGD with Nesterov’s Accelerated Gradient Method

Despite the improvement of HB over vanilla SGD, the HB method may exhibit instability and may fail to achieve global convergence for general convex problems. Nesterov’s Accelerated Gradient method (NAG) [15] improves upon HB by introducing a look-ahead step in the direction of the previous momentum. The gradient is computed at this new position, and then the parameters are updated. The iterative procedure is given by

$$y_t = x_t + \beta(x_t - x_{t-1}), \quad (15)$$

$$x_{t+1} = y_t - \eta_t g(y_t, \xi_t), \quad (16)$$

where the gradient is evaluated at the look-ahead point y_t . This correction step anticipates the future position of the iterate, leading to improved stability and accelerated convergence behavior. In the deterministic setting, NAG achieves a convergence rate of $O(1/T^2)$ for convex functions, compared to $O(1/T)$ for HB, which is optimal among first-order methods [15]. It should be noted that in the stochastic setting, the acceleration benefit of NAG is more nuanced and depends on the noise regime of the objective function or gradients [1].

4 Adaptive Learning Rate Methods

For ease of implementation, classical stochastic optimization often assumes a predetermined learning rate sequence. However, learning rate selection is often problematic: if η_t is set too large, the algorithm may become unstable or diverge, whereas if the learning rate parameter is chosen too small, convergence may be too slow or local convergence may occur. As a result, modifications have been made to (7) to alleviate the difficulty of manual parameter tuning. We present several adaptive learning rate methods, which modify the step size at each iteration based on gradient history. For the following methods, we again assume $g_t := g(x_t, \xi_t)$ is the unbiased gradient estimator of $\nabla f(x_t)$ such that (6) holds.

4.1 AdaGrad

The AdaGrad (Adaptive Gradient Algorithm) [4] optimizer adjusts the learning rate for each parameter using the accumulated sum of past squared gradients. In component-wise form, the updates are scaled by

$$x_{t+1} = x_t - \frac{\eta}{\sqrt{\sum_{i=1}^t g_i^2}} g_t. \quad (17)$$

Typically, a small constant $\varepsilon > 0$ is added for numerical stability:

$$x_{t+1} = x_t - \frac{\eta}{\sqrt{\sum_{i=1}^t g_i^2 + \varepsilon}} g_t. \quad (18)$$

AdaGrad is ideal for problems where vanishing and exploding gradient estimators heavily influence the direction of the algorithm. However, a disadvantage of this optimizer is that as the training progresses, the learning rate can decay too quickly, slowing convergence in later iterations or potentially causing premature convergence.

4.2 RMSProp

RMSProp (Root Mean Square Propagation) [24, 13] improves upon AdaGrad’s rapidly diminishing learning rates by introducing a moving average of squared gradients to normalize the gradient updates. The framework for this optimizer requires two recursions:

$$\nu_t = \gamma\nu_{t-1} + (1 - \gamma)g_t^2, \quad (19)$$

$$x_{t+1} = x_t - \frac{\eta}{\sqrt{\nu_t}}g_t, \quad (20)$$

where γ is the decay rate³. RMSProp ensures that the updates are scaled appropriately for each iterate, and ultimately achieves better performance for higher iterations, without drastically slowing down convergence.

4.3 Adam

The Adam method [12] was designed to combine the advantages offered by the AdaGrad and RMSProp methods, yielding an algorithm that works well with noisy gradients and non-stationary objective functions. This method computes individual learning rates for different parameters from first and second moments estimates of the gradient:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)g_t \quad (21)$$

$$\nu_t = \beta_2 \nu_{t-1} + (1 - \beta_2)g_t^2 \quad (22)$$

$$x_{t+1} = x_t - \eta \frac{m_t}{\sqrt{\nu_t} + \epsilon} \quad (23)$$

where m_t and ν_t represent the first and second moments, respectively. The algorithm updates the exponential moving averages of the gradient (m_t) and the squared gradient (ν_t), where hyper-parameters $\beta_1, \beta_2 \in [0, 1]$ control the exponential decay rates of these moving averages⁴. However, since the moment estimates are initialized as zeros, they are thus biased towards zero, especially during the initial time-steps and when the decay rates are small (i.e. the β s are close to 1). The initialization bias can be easily corrected using bias-corrected estimates $\hat{m}_t$ and $\hat{\nu}_t$:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{\nu}_t = \frac{\nu_t}{1 - \beta_2^t}, \quad (24)$$

such that the update rule becomes

$$x_{t+1} = x_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{\nu}_t} + \epsilon}. \quad (25)$$

The advantage of Adam lies in the algorithm’s computational efficiency and low-memory requirements, making its implementation a suitable choice for problems with large data or multiple parameters. This method, along with modern variants (AdaMax, AdamW), is currently the standard choice of optimizer in deep learning settings, popular in training neural networks. However, in settings that require a diverse set of deep learning tasks, Adam optimizers sometimes have low generalizing performance. In some cases, using vanilla SGD alongside Adam can yield better generalizing performance than Adam alone.

³The default parameter is $\gamma = 0.9$ in Keras/MATLAB and $\gamma = 0.99$ in PyTorch. The decay rate balances the contribution of past and recent gradients.

⁴In PyTorch, the default parameters are $\beta_1 = 0.9$, $\beta_2 = 0.99$.

5 Variance Reduction Methods

Variance reduction methods are an additional tool for improving convergence rates of stochastic optimization algorithms. Previously, we discussed momentum methods, which achieve implicit variance reduction as a consequence of the algorithm design. In this section, we examine explicit variance reduction methods, namely extensions of SGD that construct lower-variance gradient estimators through their update rules or simulation techniques. These methods aim to reduce the variance of our stochastic gradient estimator g_t so that it more accurately approximates the full gradient $\nabla f(x_t)$. In particular, many variance reduction methods are designed such that

$$\mathbb{E}[\|g_t - \nabla f(x_t)\|^2] \rightarrow 0$$

as $t \rightarrow \infty$ under appropriate assumptions [8]. This property enables stronger convergence guarantees and ensures that the algorithm stabilizes as it approaches an optimal solution x^* .

5.1 Finite-Sum Methods

In the classical stochastic approximation setting, the objective function is expressed as an expectation. In contrast, stochastic variance reduction methods exploit an underlying finite-sum structure, where the objective takes the form

$$f(x) = \frac{1}{n} \sum_{i=1}^n F_i(x), \quad (26)$$

such that the minimization problem (5) can be written as

$$\min_x \left\{ f(x) = \frac{1}{n} \sum_{i=1}^n F_i(x) \right\}.$$

While minimizing an expectation and a finite average are mathematically equivalent, a finite-sum formulation specifies that our objective function is composed of a fixed, finite collection of estimates, which can be exploited algorithmically. This construction differs from classical SGD and its variants, where stochastic gradients are obtained as independent noisy samples per iteration.

Assume each F_i corresponds to a loss associated with a single data point. Suppose F_i is an L -smooth function, meaning its gradient is Lipschitz continuous:

$$\|\nabla F_i(x) - \nabla F_i(x')\| \leq L\|x - x'\|$$

for all $x, x' \in \mathbb{R}^d$ and some constant $0 < L < \infty$. Under this structure, stochastic gradients are formed by sampling an index $k \in \{1, \dots, n\}$ uniformly at random and computing

$$g_t := \nabla F_k(x_t),$$

which serves as an unbiased estimator of the full gradient $\nabla f(x_t)$. The resulting SGD procedure takes the standard form

$$x_{t+1} = x_t - \eta_t g_t, \quad (27)$$

where at each iteration, the gradient estimator is constructed from a single randomly selected component F_k , rather than the full objective. Unlike adaptive learning rate or momentum-based methods, which aim to improve convergence through modifications of the update rule, finite-sum methods instead exploit the structure of $f(x)$ itself. In particular, since the same n component functions are reused across iterations, information from previously computed gradients can be stored and recycled to construct more accurate estimates of $\nabla f(x_t)$, reducing the variance in g_t .

5.1.1 SAG

The stochastic average gradient (SAG) method [23] was one of the first major approaches to reducing variance in SGD. In stochastic optimization, we aim to find an optimal x^* that minimizes the objective function $f(x)$, equivalently satisfying $\nabla f(x^*) = 0$. Since $\nabla f(x)$ can be written as the average of component gradients

(6), this motivates constructing gradient estimates using stored information. The SAG algorithm maintains a collection (or “table”) of stored gradients $\{g_t^{(i)}\}_{i=1}^n$, requiring $O(nd)$ memory to store one gradient per component function. Suppose we initialize $\{g_0^{(i)}\}_{i=1}^n$. The update is given by

$$x_{t+1} = x_t - \eta_t \cdot \frac{1}{n} \sum_{i=1}^n g_t^{(i)}, \quad (28)$$

where at each iteration t , an index $k \in \{1, \dots, n\}$ is sampled uniformly at random and the stored gradients are updated according to

$$g_t^{(i)} = \begin{cases} \nabla F_k(x_t), & \text{if } i = k, \\ g_{t-1}^{(i)}, & \text{if } i \neq k. \end{cases} \quad (29)$$

Thus, only a single component gradient is updated at each iteration, while the others remain unchanged. The search direction is given by the average of these stored gradients, yielding a low-variance approximation of the full gradient. In this way, SAG approximates the full gradient without recomputing all component gradients at every step.

5.1.2 SAGA

A practical issue that arises with SAG is that the resulting gradient estimator is biased, which complicates theoretical analysis despite the method’s strong empirical performance. The SAGA method [3] is a modification of SAG that uses control variates to produce low-variance unbiased estimators of the gradient. As in SAG, a collection of stored gradient estimates $\{g_t^{(i)}\}_{i=1}^n$ is maintained at $O(nd)$ memory cost. At each iteration, an index $k \in \{1, \dots, n\}$ is sampled uniformly at random and a new gradient is computed as

$$g_{t+1} := \nabla F_k(x_t).$$

The update rule takes the form

$$x_{t+1} = x_t - \eta_t \left(g_{t+1} - g_t^{(k)} + \frac{1}{n} \sum_{i=1}^n g_t^{(i)} \right), \quad (30)$$

and the stored gradients are updated as

$$g_{t+1}^{(i)} = \begin{cases} g_{t+1}, & \text{if } i = k, \\ g_t^{(i)}, & \text{if } i \neq k. \end{cases} \quad (31)$$

The term $g_{t+1} - g_t^{(k)}$ removes the outdated gradient information for the sampled component, while the average term incorporates information from all components. This construction yields an unbiased estimator of the full gradient with reduced variance, improving upon the flexibility and performance of SAG. The update direction satisfies

$$\mathbb{E} \left[g_{t+1} - g_t^{(k)} + \frac{1}{n} \sum_{i=1}^n g_t^{(i)} \right] = \nabla f(x_t),$$

ensuring the unbiasedness of the gradient estimator.

5.1.3 SVRG

The stochastic variance reduced gradient (SVRG) method [10] improves upon the aforementioned methods without the need for heavy storage requirements. The algorithm operates in epochs of m iterations⁵. At the start of each epoch, the current point is saved as a snapshot $\tilde{x}$, and the full gradient is computed:

$$\tilde{\mu} = \nabla f(\tilde{x}) = \frac{1}{n} \sum_{i=1}^n \nabla F_i(\tilde{x}). \quad (32)$$

⁵One complete “loop” of SVRG, where the full gradient is computed, occurs after m updates of the stored gradient components (i.e. an epoch encompasses m iterations of the algorithm).

At each iteration within the epoch, an index $k \in \{1, \dots, n\}$ is sampled uniformly at random and the gradients are computed as

$$g_t := \nabla F_k(x_t), \quad \tilde{g}_k := \nabla F_k(\tilde{x}).$$

The update rule is given by

$$x_{t+1} = x_t - \eta_t (g_t - \tilde{g}_k + \tilde{\mu}), \quad (33)$$

where the term $g_t - \tilde{g}_k$ acts as a control variate, correcting the stochastic gradient using information from the snapshot point. This yields an unbiased estimator satisfying

$$\mathbb{E}[g_t - \tilde{g}_k + \tilde{\mu}] = \nabla f(x_t).$$

Unlike SAG and SAGA, SVRG avoids storing a full table of gradients, trading increased computation (from periodic evaluations of the full gradient) for reduced memory requirements. Additionally, unlike vanilla SGD, the learning rate parameter for SVRG is not required to decay to zero, allowing for larger step sizes and faster convergence in practice.

5.1.4 SARAH

A modern approach to finite-sum minimization problems is the stochastic recursive gradient algorithm (SARAH) [16]. Like SVRG, SARAH does not require the storage of past gradients. Specifically, the SARAH method updates the stochastic step direction recursively by adding and subtracting component gradients. Additionally, the variance of the recursive gradient estimator decreases along the trajectory, allowing for improved stability compared to standard stochastic gradient methods. The estimator is initialized using the full gradient:

$$\hat{g}_0 = \nabla f(x_0).$$

At each iteration, we choose an index $k \in \{1, \dots, n\}$ uniformly at random, and compute the gradients:

$$g_t := \nabla F_k(x_t), \quad g_{t-1} := \nabla F_k(x_{t-1}).$$

A new estimator for the gradient is then constructed recursively as

$$\hat{g}_t = \hat{g}_{t-1} + g_t - g_{t-1}.$$

The update is given by

$$x_{t+1} = x_t - \eta_t \hat{g}_t, \quad (34)$$

where $\hat{g}_t$ is a biased estimator of the gradient. SARAH requires two component gradient evaluations per iteration, but does not store per-sample gradients, resulting in lower memory cost than the SAG/SAGA methods. It can be shown that SARAH admits linear convergence for strongly convex functions [16].

5.2 Importance Sampling

In this section, we detail a theoretical approach for reducing variance in stochastic estimators. Unlike finite-sum variance reduction methods, which exploit the structure of the objective function, sampling-based methods reduce variance by modifying the distribution from which stochastic gradients or observations are drawn. Particularly, importance sampling in stochastic optimization is a variance reduction technique which emphasizes random samples that have more impact on the loss function. The idea is to choose an alternative sampling distribution that allows “important” values to be drawn more frequently, and then re-weight the estimates to correct for sampling from a biased distribution. This is especially useful in problems involving rare events, where the expected loss function may be dominated by extreme outcomes that are infrequently observed under the original sampling distribution.

Suppose the gradient estimator $g_t := \nabla F(x_t, \xi_t)$ is constructed with random samples ξ_t from sampling distribution $\mathbb{P}$ with density p . Let $\mathbb{Q}$ denote an alternative sampling distribution with density q , and let $\frac{p(\xi)}{q(\xi)}$

denote the likelihood ratio (Radon–Nikodym derivative). Then the stochastic optimization problem (5) can be written as [25]:

$$f(x^*) = \min_x \mathbb{E}_{\xi \sim \mathbb{P}}[F(x, \xi)] = \min_x \mathbb{E}_{\xi \sim \mathbb{Q}} \left[F(x, \xi) \frac{p(\xi)}{q(\xi)} \right]. \quad (35)$$

The corresponding importance sampling gradient estimator is given by

$$g_t := \nabla F(x_t, \xi_t) \frac{p(\xi_t)}{q(\xi_t)}, \quad \xi_t \sim \mathbb{Q}, \quad (36)$$

which satisfies

$$\mathbb{E}_{\xi_t \sim \mathbb{Q}}[g_t] = \nabla f(x_t). \quad (37)$$

Thus, g_t remains an unbiased estimator of the full gradient. The choice of the sampling distribution $\mathbb{Q}$ is crucial, as an effective choice of $\mathbb{Q}$ will place more weight on samples that contribute significantly to the gradient, thereby reducing variance and improving convergence.

5.2.1 Exponential Tilting

One approach to selecting an importance sampling distribution is to modify the original sampling distribution through exponential tilting. This technique is often referred to as exponential twisting or exponential change of measure in the literature.

Given a random variable ξ with distribution $\mathbb{P}$, density function p , and moment-generating function $M_\xi(\theta) = \mathbb{E}[e^{\theta\xi}]$, we define the tilted density p_θ by

$$p_\theta(\xi) = \frac{e^{\theta\xi} p(\xi)}{M_\xi(\theta)}, \quad (38)$$

where θ is the tilting parameter. This transformation shifts probability mass toward larger values of ξ when $\theta > 0$, making rare but important events more likely to be observed in simulation. The corresponding likelihood ratio used to correct for this change of measure is given by

$$\frac{p(\xi)}{p_\theta(\xi)} = \frac{M_\xi(\theta)}{e^{\theta\xi}}. \quad (39)$$

In the final section, we investigate the effectiveness of importance sampling (with exponential tilting) in minimizing the variance of a sampling estimator. Specifically, we examine the effectiveness of coupling variance reduction with varying stochastic optimizers to efficiently price a financial derivative.

6 Barrier Option Pricing

Down-and-in barrier options are a type of path-dependent derivative that become “knocked-in” (activated) when the underlying asset crosses a prespecified threshold during the option’s lifetime. Let $S(t)$ denote the underlying asset at time t and suppose a down-and-in European call is written on $S(t)$ with strike K , barrier B , and expiration T . The discounted expected payoff for this derivative is given by

$$V = \mathbb{E}[e^{-rT}((S(T) - K)^+ \cdot \mathbf{1}_{\min_{1 \leq i \leq n} S(t_i) < B})], \quad (40)$$

where the option is observed over the discrete time grid $t_1, t_2, \dots, t_n = T$ and e^{-rT} is the discount factor with respect to risk-free interest rate r .

A typical approach to pricing a barrier option is crude Monte Carlo, which estimates this expectation by simulating N stock price paths and averaging the payoffs. However, if $S(0) \gg B$, brute force random sampling will yield few paths that cross the barrier, meaning few paths will return non-zero option prices, resulting in high variance estimates. This motivates the use of importance sampling techniques. The idea is to gather more useful information from stock price paths by modifying the probability distribution of the underlying process to increase the probability that the stock crosses B and expires above strike K .

6.1 Importance Sampling Framework

Let $t_i = ih$ be a uniform time discretization. Consider a general asset price model of the form

$$S(t_i) = S(0) \exp(L_i),$$

where $L_i = \sum_{j=1}^i X_j$ with $L_0 = 0$ and $X_1, X_2, \dots$ are i.i.d. random variables with density p . We assume a standard GBM model such that $X_i \sim \mathcal{N}((r - \frac{1}{2}\sigma^2)h, \sigma^2h)$. Observe that

$$S(T) > K \iff S(0) \exp(L_n) > K \iff L_n > \log(K/S(0))$$

and

$$\min_{1 \leq i \leq n} S(t_i) < B \iff \min_{1 \leq i \leq n} S(0) \exp(L_i) < B \iff \min_{1 \leq i \leq n} L_i < \log(B/S(0)).$$

Set $k = \log(K/S(0))$ and $b = \log(B/S(0))$. Suppose τ denotes the first time L_i drops below b . Then $L_\tau = \log(S(t_\tau)/S(0)) < b$ and the barrier option payoff function (40) becomes

$$(S(0)e^{L_n} - K)^+ \mathbf{1}_{\{L_n > k, \tau \leq n\}}. \quad (41)$$

The importance sampling approach is to construct two new density functions, $p_{\theta^+}(x)$ and $p_{\theta^-}(x)$ to simulate the stock price. First, we aim to twist the density such that the stock price paths go down to the barrier faster. Until the barrier is crossed, we simulate $X_1, X_2, \dots, X_\tau$ from

$$p_{\theta^-}(x) = \frac{e^{\theta^- x} p(x)}{M(\theta^-)}, \quad (42)$$

where twisting parameter θ^- is chosen such that the drift is negative and $p_{\theta^-}(x)$ is the density function for $\mathcal{N}((r - \sigma^2/2)h + \sigma^2 h \theta^-, \sigma^2 h)$. After the barrier is crossed, we now wish to shift the price paths so that they go up to the exercise price faster. We take $X_{\tau+1}, X_{\tau+2}, \dots, X_n$ from

$$p_{\theta^+}(x) = \frac{e^{\theta^+ x} p(x)}{M(\theta^+)}, \quad (43)$$

where θ^+ is chosen such that the drift is positive and $p_{\theta^+}(x)$ is the density for $\mathcal{N}((r - \sigma^2/2)h + \sigma^2 h \theta^+, \sigma^2 h)$. For both densities, $M(\theta)$ is the moment-generating function. With these twisted density choices, we obtain the following likelihood ratio weights:

$$\begin{aligned} W_{\theta^-, \theta^+} &= \prod_{i=1}^{\tau} \frac{p(X_i)}{p_{\theta^-}(X_i)} \prod_{i=\tau+1}^n \frac{p(X_i)}{p_{\theta^+}(X_i)} \\ &= \frac{M(\theta^-)^\tau}{\exp(\theta^- (X_1 + \dots + X_\tau))} \frac{M(\theta^+)^{n-\tau}}{\exp(\theta^+ (X_{\tau+1} + \dots + X_n))} \\ &= M(\theta^-)^\tau M(\theta^+)^{n-\tau} \exp(-\theta^- L_\tau - \theta^+ (L_n - L_\tau)). \end{aligned}$$

If we assume our stock price follows a standard geometric Brownian motion (GBM), then $p(x)$ in the above equations is the normal density $\mathcal{N}((r - \sigma^2/2)h, \sigma^2 h)$, and thus p_{θ^-} and p_{θ^+} are also normal densities. There exist known heuristics (discussed in detail in [17, 6, 2]) for the selection of θ^+ and θ^- :

$$\theta^- = \left(\frac{1}{2} - \frac{r}{\sigma^2} \right) + \left(\frac{2b + k}{n\sigma^2 h} \right), \quad \theta^+ = \left(\frac{1}{2} - \frac{r}{\sigma^2} \right) - \left(\frac{2b + k}{n\sigma^2 h} \right). \quad (44)$$

In particular, if θ^- and θ^+ are chosen as in (44) so that $M(\theta^-) = M(\theta^+)$, the likelihood ratio simplifies to

$$W_{\theta^-, \theta^+} = M(\theta^+)^n \exp((\theta^+ - \theta^-)L_\tau - \theta^+ L_n).$$

Therefore, the importance sampling estimator for the down-and-in call price becomes

$$\hat{V} = e^{-rT} (S(0)e^{L_n} - K)^+ \mathbf{1}_{\{\tau \leq n\}} \cdot W_{\theta^-, \theta^+}. \quad (45)$$

While these heuristic choices for θ^+, θ^- perform well, they are not guaranteed to be optimal. In this work, we instead use the heuristics as baselines and estimate (θ^-, θ^+) by minimizing the second moment of the estimator:

$$f(\theta^-, \theta^+) = \mathbb{E}_{\mathbb{Q}_{\theta^-, \theta^+}} [\hat{V}^2]. \quad (46)$$

Since this expectation is not available in closed form, we approximate it using Monte Carlo. This defines a stochastic optimization problem of the form (5), where the objective is an expectation that cannot be evaluated exactly but can be estimated via simulation. We initialize (θ_0^-, θ_0^+) using the heuristics (44), which place the starting parameters in a reasonable region and reduce the number of iterations required to reach the optimum. Our goal is to investigate the efficiency and accuracy of stochastic optimization methods in estimating the optimal twisting parameters. Formally, the stochastic optimization approach aims to solve

$$(\theta^{*-}, \theta^{*+}) = \arg \min_{\theta^-, \theta^+} \mathbb{E}_{\mathbb{Q}_{\theta^-, \theta^+}} [\hat{V}^2]. \quad (47)$$

To apply stochastic optimization methods, we need to construct an unbiased gradient estimator of ∇f . A convenient form is given by taking the gradient of the log-likelihood function:

$$\nabla f(\theta^-, \theta^+) = \mathbb{E}_{\mathbb{Q}_{\theta^-, \theta^+}} [\hat{V}^2 \cdot \nabla \log W_{\theta^-, \theta^+}].$$

Using the simplified likelihood ratio, we obtain

$$\log W_{\theta^-, \theta^+} = \tau \log M(\theta^-) + (n - \tau) \log M(\theta^+) - \theta^- L_\tau - \theta^+ (L_n - L_\tau),$$

which yields

$$\begin{aligned} \frac{\partial}{\partial \theta^-} \log W_{\theta^-, \theta^+} &= \tau \frac{M'(\theta^-)}{M(\theta^-)} - L_\tau, \\ \frac{\partial}{\partial \theta^+} \log W_{\theta^-, \theta^+} &= (n - \tau) \frac{M'(\theta^+)}{M(\theta^+)} - L_n + L_\tau. \end{aligned}$$

Therefore, for a single simulated path, we use the stochastic gradient estimator

$$\widehat{\nabla} f(\theta^-, \theta^+) = \hat{V}^2 \begin{pmatrix} \tau \frac{M'(\theta^-)}{M(\theta^-)} - L_\tau \\ (n - \tau) \frac{M'(\theta^+)}{M(\theta^+)} - L_n + L_\tau \end{pmatrix}. \quad (48)$$

For standard GBM, the moment-generating function is

$$M(\theta) = \exp \left(\left(r - \frac{1}{2} \sigma^2 \right) h \theta + \frac{1}{2} \sigma^2 h \theta^2 \right),$$

so that the ratio in (48) simplifies to

$$\frac{M'(\theta)}{M(\theta)} = \left(r - \frac{1}{2} \sigma^2 \right) h + \sigma^2 h \theta.$$

We will approximate the gradient using a simulated paths, and update the parameters using various stochastic optimization schemes of the following form:

$$(\theta_{t+1}^-, \theta_{t+1}^+) = (\theta_t^-, \theta_t^+) - \eta_t \widehat{\nabla} f(\theta_t^-, \theta_t^+).$$

6.2 Numerical Results

We evaluate the performance of the vanilla SGD, SGDM, and Adam optimizers in estimating the optimal twisting parameters $(\theta^{*-}, \theta^{*+})$ for the down-and-in barrier call option described in the previous section. The model parameters are set as follows: initial asset price $S_0 = 110$, strike $K = 100$, barrier $B = 90$, risk-free rate $r = 0.05$, volatility $\sigma = 0.4$, maturity $T = 1.0$, and $n = 52$ discrete time steps (i.e. weekly observations).

The asset price is assumed to follow a standard GBM. Each optimizer is initialized at the heuristic parameters (44) and run for 1,000 iterations with one simulated path per gradient update⁶. Variance estimates are then computed at the final parameter values, using $N = 10,000$ independent Monte Carlo replications.

Under this model, the heuristic twisting parameters are given by $\theta^- = -2.9166$ and $\theta^+ = 3.2916$. The optimized parameters obtained from each method are given in Table 1 and the parameter optimization paths are given in Figure 1.

Method	θ^-	θ^+
Heuristic	-2.9166	3.2916
SGD	-2.9192	3.2969
SGDM	-2.9466	3.3397
Adam	-3.3886	3.7103

Table 1: Optimized twisting parameters obtained from each method, initialized at the heuristic values.

SGD and SGDM converge to parameters close to the heuristic values, while Adam moves farther away from the initialization, suggesting that the adaptive learning rate allows it to explore a larger region of the parameter space within the same number of iterations.

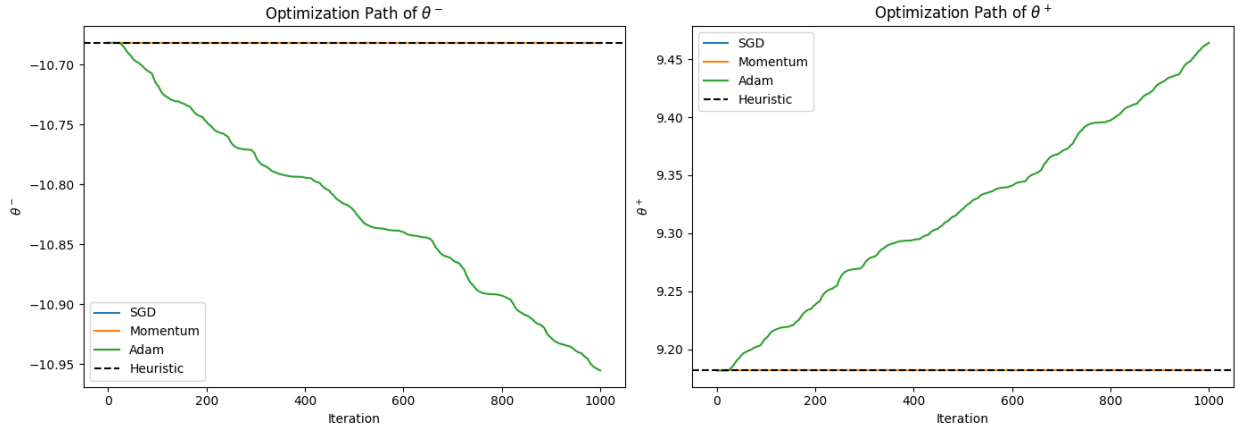


Figure 1: Optimization paths of the twisting parameters θ^- (left) and θ^+ (right) over 1,000 iterations for each method, initialized at the heuristic values (dashed line).

The plots further display the immediate convergence of SGD and SGDM to values near the heuristic parameters, while Adam moves progressively further from the initialization. The continued drift of Adam's path for θ^+ at iteration 1,000 suggests the method did not fully converge within the given number of iterations.

To assess the quality of the final parameter estimates, we compute the variance of the importance sampling estimator $\hat{V}$ at the final parameter values for each method, using $N = 10,000$ independent Monte Carlo replications. The results are displayed in Figure 2.

All three optimizers reduce the estimator variance relative to the heuristic baseline of 1.95×10^{-4} . Adam achieves the largest variance reduction, yielding a variance of 1.66×10^{-4} , which is a 14.9% improvement over the heuristic. SGD and SGDM achieve more modest reductions to 1.85×10^{-4} and 1.82×10^{-4} respectively, consistent with their convergence to parameter values close to the heuristic initialization. These results confirm that stochastic optimization can improve upon the heuristic twisting parameters, and that Adam's ability to adapt its learning rate per parameter allows it to identify a lower variance region that SGD and SGDM do not reach within the same iteration budget.

⁶Stronger results can be obtained using a batch of simulated paths per iteration.

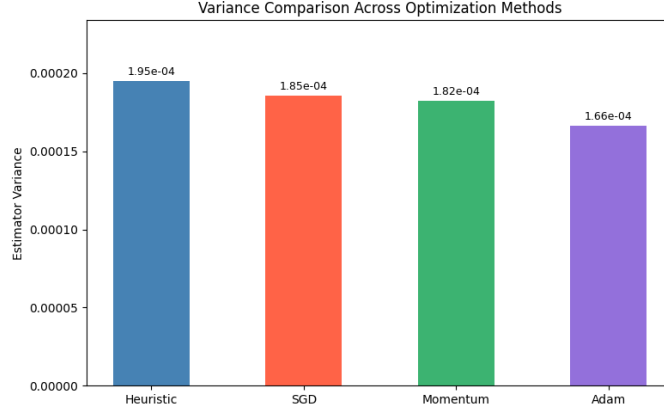


Figure 2: Estimator variance at the final twisting parameters obtained from each optimization method, computed from $N = 10,000$ independent Monte Carlo replications.

References

- [1] M. Assran and M. Rabbat. On the Convergence of Nesterov’s Accelerated Gradient Method in Stochastic Settings, in *Proceedings of the 37th International Conference on Machine Learning*, 2020.
- [2] P. Boyle, M. Broadie, P. Glasserman. Monte Carlo methods for security pricing. *Journal of Economic Dynamics and Control* 21, (1997) 1267–1321.
- [3] A. Defazio, F. Bach, and S. Lacoste-Julien. SAGA: A Fast Incremental Gradient Method With Support for Non-Strongly Convex Composite Objectives, in *Advances in Neural Information Processing Systems*, 2014. <https://arxiv.org/abs/1407.0202>
- [4] J. Duchi, E. Hazan, and Y. Singer. Adaptive Subgradient Methods for Online Learning and Stochastic Optimization, *Journal of Machine Learning Research*, 12:2121–2159, 2011.
- [5] V. Fabian. On Asymptotic Normality in Stochastic Approximation. *The Annals of Mathematical Statistics*, 39(4), 1327–1332, 1968. <http://www.jstor.org/stable/2239701>
- [6] P. Glasserman. Monte Carlo Methods in Financial Engineering. *Springer*, 2003.
- [7] L. Goldstein. On the Choice of Step Size in the Robbins–Monro Procedure, *Statistics & Probability Letters*, 6:299–303, 1988.
- [8] R. M. Gower, M. Schmidt, F. Bach, and P. Richtárik. Variance-Reduced Methods for Machine Learning, arXiv preprint, 2020. <https://arxiv.org/abs/2010.00892>
- [9] B. Heidergott and F. Vázquez-Abad. Optimization and Learning via Stochastic Gradient Search, *Princeton Series in Applied Mathematics*, 2025.
- [10] R. Johnson and T. Zhang. Accelerating Stochastic Gradient Descent using Predictive Variance Reduction, in *Advances in Neural Information Processing Systems*, 26:315–323, 2013.
- [11] J. Kiefer and J. Wolfowitz. Stochastic Estimation of the Maximum of a Regression Function, *The Annals of Mathematical Statistics*, 23:462–466, 1952.
- [12] D. P. Kingma and J. Ba. Adam: A Method for Stochastic Optimization, *arXiv preprint arXiv:1412.6980*, 2014. <https://arxiv.org/abs/1412.6980>
- [13] J. Lu. Gradient Descent, Stochastic Optimization, And Other Tales, <https://arxiv.org/pdf/2205.00832v2>, 2024.

- [14] A. Nemirovski, A. Juditsky, G. Lan, and A. Shapiro. Robust Stochastic Approximation Approach to Stochastic Programming, *SIAM Journal on Optimization*, 19:1574–1609, 2009.
- [15] Y. Nesterov. A Method for Solving the Convex Programming Problem with Convergence Rate $O(1/k^2)$, *Doklady Akademii Nauk SSSR*, 269:543–547, 1983.
- [16] L. M. Nguyen, J. Liu, K. Scheinberg, and M. Takáč. SARAH: A Novel Method for Machine Learning Problems Using Stochastic Recursive Gradient, in *Proceedings of the 34th International Conference on Machine Learning*, 2017. <https://proceedings.mlr.press/v70/nguyen17b.html>
- [17] G. Okten. Monte Carlo and Quasi-Monte Carlo Methods in Computational Finance. Draft, 2026.
- [18] B. T. Polyak. Some Methods of Speeding Up the Convergence of Iteration Methods, *USSR Computational Mathematics and Mathematical Physics*, 4:1–17, 1964.
- [19] B. T. Polyak. New Method of Stochastic Approximation Type, *Automation and Remote Control*, 51:937–1008, 1990.
- [20] B. T. Polyak and A. Juditsky. Acceleration of Stochastic Approximation by Averaging, *SIAM Journal on Control and Optimization*, 30(4):838–855, 1992.
- [21] H. Robbins and S. Monro. A Stochastic Approximation Method, *The Annals of Mathematical Statistics*, 22:400–407, 1951.
- [22] D. Ruppert. Efficient Estimators from a Slowly Convergent Robbins–Monro Process, Technical Report No. 781, Cornell University, 1988.
- [23] M. Schmidt, N. Le Roux, and F. Bach. Minimizing Finite Sums with the Stochastic Average Gradient, *Mathematical Programming*, 162:83–112, 2015. <https://arxiv.org/abs/1309.2388>
- [24] T. Tieleman and G. Hinton. Lecture 6.5-rmsprop: Divide the Gradient by a Running Average of Its Recent Magnitude. *COURSERA: Neural Networks for Machine Learning*, 4:26–31, 2012.
- [25] L. Aolaritei, B. P. G. Van Parys, H. Lam, and M. I. Jordan. Stochastic Optimization with Optimal Importance Sampling, arXiv preprint, 2025. <https://arxiv.org/abs/2504.03560>
- [26] F. Vázquez-Abad and B. Heidergott. Optimization and Learning via Stochastic Gradient Search, *Princeton Series in Applied Mathematics*, 2025.
- [27] Z. Wang and A. Yurtsever. Generalized Stochastic Gradient Descent with Momentum Methods for Smooth Optimization, preprint, 2026.